Phase A: OWASP Top 10 Summary

01. Broken Access Control

Broken Access Control occurs when an application fails to enforce the permissions assigned to users, allowing them to access information or perform actions beyond what they are authorized to do. This can include accessing another user's data by modifying an identifier, accessing administrative functions, bypassing API authorization checks, or gaining higher privileges. A common mistake is relying on the front-end to enforce authorization, since an attacker can directly send requests to the backend. To prevent this, access control must be enforced server-side, with access denied by default, proper ownership and role checks, logging of access-control failures, and appropriate session/token management.

Real world example: In 2019, Facebook addressed a major data exposure risk tied to broken access control and improper authorization in its contact importer tool.

Facebook's contact importer feature (designed to let users sync and find friends via phone contacts) lacked robust rate-limiting and authorization checks on bulk requests.

Attackers fed large lists of sequential or generated phone numbers into the contact import endpoint. Because access controls failed to properly restrict query frequencies or validate authorization for mass enumeration, the API returned matching user profiles and associated data.

This led to massive unauthorized data harvesting (scraping), exposing phone numbers, Facebook IDs, and names for hundreds of millions of users globally. The compiled datasets later surfaced publicly on hacker forums in 2021, prompting widespread regulatory and privacy scrutiny.

02. Security Misconfiguration

Every application or service need to be set up correctly to avoid vulnerabilities. Security misconfiguration is when the application or the service is not set up correctly, thus leading to vulnerabilities. Keeping the application or the service with its default parameters after installation, keeping unnecessary features enabled and leaving default user account password unchanged can lead to vulnerabilities. Misconfiguration can happen at multiple layers (application, web server, database, cloud infrastructure, container, framework, etc.). To prevent it, we must implement secure installation process by automating repeatable hardening processes to create a secured environment, keeping only necessary features for a platform. Secrets should be stored and managed using appropriate secret-management mechanisms rather than hardcoded in source code.

Real word example: March 2019 Capital One data breach structured around its security misconfigurations.

Capital One deployed an open-source firewall instance (ModSecurity) to protect its web application on Amazon Web Services (AWS). However, the firewall was misconfigured with excessive routing permissions, allowing it to relay unauthorized requests directly to the internal network. The WAF was granted a highly privileged Identity and Access Management (IAM) role. Instead of following the principle of least privilege, this single firewall profile possessed broad, unrestricted capabilities to read data across all of Capital One's storage components.

The attacker exploited the WAF's open routing to launch an SSRF attack. She manipulated the firewall server into making a localized request to the AWS Instance Metadata Service (IMDSv1) at the default IP 169.254.169.254. Because IMDSv1 lacked session-oriented defenses, the compromised WAF instantly returned the temporary security credentials belonging to its highly privileged IAM role. Equipped with these stolen administrative credentials, the attacker bypassed external security boundaries. She executed command-line scripts to list, read, and clone hundreds of proprietary data store structures.

The breach exposed the personal data of approximately 106 million individuals across the United States and Canada. The stolen cache totaled over 30 gigabytes of data, containing credit scores, income structures, credit limits, 140,000 Social Security numbers, and 80,000 linked bank account numbers. The US Office of the Comptroller of the Currency (OCC) hit Capital One with an $80 million civil penalty for failing to establish effective cloud risk assessment structures. Additionally, the bank paid a $190 million class-action settlement to resolve lawsuits filed by affected consumers.

03. Software Supply Chain Failures

Software Supply Chain is the use of third-party code, tools or other dependencies that the system relies on. Vulnerabilities in this supply chain can lead the system to be vulnerable. System might be vulnerable if we don’t track version of dependencies included in our system, meaning, identify vulnerable versions of the dependencies being used. It might vulnerable if we don’t scan for vulnerabilities regularly. It might also be vulnerable if we don’t fix or upgrade the dependencies we use. To prevent it, we must track direct dependencies and their transitive dependencies, remove unused dependencies, continuously monitor for vulnerabilities on official sources like CVE, NVD, OSV, etc and update regularly CI/CD tools.

Real world example: 2017 Equifax data breach structured around its unpatched vulnerability and architectural failures.

The breach targeted Equifax's online customer dispute portal, which ran on Apache Struts, an open-source web application framework. A critical Remote Code Execution (RCE) vulnerability (tracked as CVE-2017-5638) was publicly disclosed on March 7, 2017. Despite receiving alerts from the Department of Homeland Security and internal security teams in early March, Equifax failed to apply the available software patch to its dispute portal system. Equifax utilized an internal network traffic inspection tool to monitor for data irregularities. However, the security certificate for this monitoring system had been expired for 10 months, meaning the company was entirely blind to malicious internal traffic.

On May 13, 2017, attackers exploited CVE-2017-5638 by sending specially crafted HTTP requests containing malicious code within the content-type headers. The vulnerable Apache Struts application executed this code, granting the attackers initial entry. The attackers dropped "web shells" (backdoors) onto the server to maintain permanent, remote administrative control over Equifax's internal network. Once inside, hackers found a file containing unencrypted database credentials (usernames and passwords), which included default configurations like "admin". Because the network lacked rigorous internal segmentation, these stolen credentials allowed them to escape the dispute portal environment and query 48 unrelated databases storing massive consumer credit histories. Attackers executed roughly 9,000 database queries, blending into normal activity. They organized data into small temporary archives, encrypted the traffic to bypass the uninspected network blocks, and exfiltrated the data gradually over a period of 76 days. The attack compromised the highly sensitive Personally Identifiable Information (PII) of 147.9 million Americans, as well as over 15 million British citizens and thousands of Canadians. Stolen records included full names, Social Security Numbers (SSNs), dates of birth, home addresses, and driver's license numbers—the exact toolkit required for lifelong identity theft. Equifax agreed to a global settlement with the Federal Trade Commission (FTC), the Consumer Financial Protection Bureau (CFPB), and all US states, paying up to $425 million (part of a broader $700 million ecosystem fallout) to fund free credit monitoring and identity theft recovery services for victims.

04. Cryptographic Failures

Cryptographic Failures occur when sensitive data is inadequately protected because encryption is missing, weak cryptographic algorithms are used, keys are poorly managed, or cryptography is incorrectly implemented. This can expose sensitive information both in transit and at rest. Passwords should not be stored using reversible encryption; they should be protected with appropriate password-hashing algorithms. Prevention includes identifying sensitive data, minimizing the amount of sensitive data

stored, using strong and well-established cryptographic algorithms, encrypting sensitive communications, and implementing proper key management.

Real world example: LinkedIn data breach in 2012

LinkedIn stored user passwords in their database using the SHA-1 cryptographic function, but without applying a "salt". SHA-1 was considered a weak and overly fast hashing algorithm for password storage. Its speed allowed attackers to compute and verify billions of potential combinations per second.

Attackers successfully breached LinkedIn's internal network and exported the database table containing user credentials. Because the hashes were unsalted, identical passwords across different accounts generated the exact same hash value. This allowed attackers to use rainbow tables to instantaneously look up and reverse millions of passwords without needing to crack each account individually.

05. Injection

Injection occurs when untrusted data is interpreted as part of a command or query instead of being treated as data. This can allow an attacker to manipulate SQL queries, operating-system commands, or other interpreters. It commonly occurs when applications construct commands or queries using untrusted input. To prevent injection, developers should use parameterized queries and safe APIs, validate input on the server side, avoid dynamically constructing commands, and apply appropriate output encoding where required.

Real world example: TalkTalk cyberattack on October 2015.

The attack targeted three outdated, legacy webpages that TalkTalk had inherited from its 2009 acquisition of internet service provider Tiscali. The underlying database software supporting these webpages was completely outdated. It contained a known security bug for which a vendor-supplied patch had been available for years, but TalkTalk had failed to apply it. The company failed to properly monitor its infrastructure. It missed two successful, smaller-scale injection attacks targeting the exact same webpages earlier that year in July and September 2015. The attackers initially launched a Distributed Denial of Service (DDoS) attack to flood TalkTalk's primary website with traffic. This served as a distraction to overwhelm the internal IT systems and mask the concurrent intrusion. With the IT team distracted, the hackers used an automated penetration testing tool called SQLMap to locate fields on the legacy pages that directly accepted user input. By entering malicious SQL commands into those web forms, they bypassed access restrictions entirely and forced the database to export sensitive customer records.

The attackers successfully exfiltrated the personal details of 156,959 customers. Stolen data included customer names, addresses, dates of birth, phone numbers, and email addresses. For 15,656 of those affected customers, the hackers gained direct access to unencrypted bank

account numbers and sort codes. This put victims at an immediate risk of highly targeted phishing scams and identity fraud.

06. Insecure Design

Secure Design is the way to design an application to be secured during the development, the deployment and the maintenance phases of the app lifecycle. It is different from secure implementation, meaning applying secure principle. The key part of having secure design is to gather requirements and resource management, create a secure design and have a secure development lifecycle. Insecure design is the failure to apply secure design. To prevent it, we have to apply a secure development lifecycle with Appsec professional, use threat modeling for critical parts of the application and apply secure implementation to the app.

07. Authentication failures

Authentication Failures occur when an application incorrectly verifies a user's identity or improperly manages authentication sessions. Attackers may exploit weak passwords, credential stuffing, brute-force attacks, missing MFA, or weaknesses in session management to gain access to another user's account. Prevention includes appropriate password policies, protection against automated authentication attacks, MFA where appropriate, secure session management, and proper handling of authentication credentials.

08. Software or Data integrity failures

Software integrity failures is the failure to ensure that the software has not been modified by a third party. Data integrity failure is the failure to ensure that the data has not been modified by a third party. To prevent it, we must use digital signatures to verify the software or the data is from the intended source, check dependencies and packages integrity. We must ensure that the CI/CD pipeline has proper segregation, signed commits/artifacts, trusted repositories, dependency integrity verification, secure build pipelines, and separation of duties.

09. Security Logging and Alerting Failures

Security logging and alerting are necessary for application security. Failure to achieve this, leads to the inability to detect attacks and breaches. To prevent it, it is necessary to

monitor all login, access control, server-side input validation failures. Ensure that monitoring stack are set up. Establish and adopt incident response and recovery plan.

10. Mishandling of Exceptional Conditions

Mishandling of Exceptional Conditions occurs when an application does not properly handle unexpected situations, errors, or abnormal states. This can result in information disclosure, inconsistent application state, security bypasses, or denial of service. Developers should identify expected failure conditions, validate inputs, handle exceptions safely, fail securely, avoid exposing sensitive error information, and use centralized exception handling where appropriate